

# Основы работы с функцией setInterval в JavaScript

В данном уроке мы научимся работать с таймерами в JavaScript. Разобравшись с таймерами, вы сможете автоматически выполнять на странице какие-либо операции через заданный промежуток времени. К примеру, можно будет сделать слайдер картинок, в котором картинки будут меняться каждую секунду.

Для работы с таймерами в JavaScript используется функция `setInterval`, которая запускает заданный код через определенные промежутки времени.

Эта функция работает следующим образом: первым параметром она принимает исходный код функции, а вторым параметром - интервал, через который эта функция будет автоматически вызываться. Вторым параметром задается в миллисекундах (1000 миллисекунд = 1 секунда).

Давайте изучим работу функции на каком-нибудь примере. Например, напомним код, который каждую секунду будет что-нибудь выводить в консоль.

Для начала сделаем функцию, выводящую что-нибудь в консоль:

```
1 function timer() {  
2   console.log('!');  
3 }
```

А теперь с помощью `setInterval` заставим созданную нами функцию выполняться каждую секунду:

```
1 setInterval(timer, 1000);  
2  
3 function timer() {  
4   console.log('!');  
5 }
```

Не обязательно создавать отдельную функцию - можно просто в первый параметр `setInterval` передать анонимную функцию, вот так:

```
1 setInterval(function() {  
2   console.log('!');  
3 }, 1000);
```

## № 1

©jsPmTrLn

Запустите таймер, который каждые 3 секунды будет что-нибудь выводить в консоль.

# Счетчик через функцию setInterval в JavaScript

Конечно же, не очень интересно то, что наш таймер каждый раз выводит одно и тоже. Давайте усложним нашу задачу и сделаем так, чтобы каждую секунду в консоль выводились числа по возрастанию: сначала 1, потом 2, потом 3 и так далее.

Для этого нам понадобится переменная-счетчик, которая будет хранить свои значения между запусками функции. Несложно понять, что можно просто сделать глобальную переменную:

```
1 let i = 0; // глобальная переменная  
2  
3 setInterval(function() {  
4   i++;  
5   console.log(i);  
6 }, 1000);
```

Перепишем более компактно:

```
1 let i = 0;  
2 setInterval(function() {  
3   console.log(++i);  
4 }, 1000);
```

Или еще более компактно через стрелочную функцию:

```
1 | let i = 0;
2 | setInterval(() => console.log(++i), 1000);
```

## № 1

©jsPmTrCn

Пусть дана переменная, в которой изначально хранится число 100. Запустите таймер, который каждую секунду будет уменьшать значение этой переменной на 1 и выводить это значение в консоль.

# Остановка таймера JavaScript

Вы уже знаете, как запустить таймер, давайте теперь научимся его останавливать. Для этого вам необходимо знать, что каждый таймер, запущенный с помощью функции `setInterval`, имеет уникальный номер. Этот номер возвращает функция `setInterval` в момент запуска таймера:

```
1 | let timerId = setInterval(function() {
2 |     console.log('!')
3 | }, 1000);
4 |
5 | alert(timerId); // выведет номер таймера
```

Для остановки таймера используется функция `clearInterval`, которая принимает уникальный номер того таймера, который нужно остановить.

Для примера давайте запустим таймер, выводящий в консоль числа по возрастанию, начиная с 1. Остановим таймер, как только на экран будет выведено число 10:

```
1 | let i = 0;
2 |
3 | let timerId = setInterval(function() {
4 |     console.log(++i);
5 |
6 |     if (i >= 10) {
7 |         clearInterval(timerId);
8 |     }
9 | }, 1000);
```

## № 1

©jsPmTrTS

Пусть дана переменная, в которой изначально хранится число 10. Запустите таймер, который каждую секунду будет уменьшать значение этой переменной на 1 и выводить это значение в консоль. Как только значение переменной достигнет нуля - остановите таймер.

# Кнопка для запуска таймера на JavaScript

В предыдущих уроках наш таймер начинал свою работу сразу по загрузке страницы. Давайте теперь сделаем кнопку, по нажатию на которую будет запускаться наш таймер:

```
1 | <button id="start">start</button>
```

Напишем соответствующий JavaScript:

```
1 | let start = document.querySelector('#start');
2 |
3 | start.addEventListener('click', function() {
4 |     let i = 0;
```

```

5
6   setInterval(function() {
7       console.log(++i);
8   }, 1000);
9 });

```

©jsPmTrLB

## № 1

Сделайте кнопку, по нажатию на которую в консоль будет выводиться обратный отсчет, начиная с 100.

# Многократное нажатие на кнопку для запуска таймера на JavaScript

Код, приведенный мною выше, на самом деле имеет некоторую проблему. Эта проблема проявляется в том случае, если на нашу кнопку сделать несколько кликов. В этом случае *каждый клик будет приводить к запуску нового таймера*.

То есть, к примеру, три нажатия на кнопку приведут к тому, что будет запущено три таймера, и каждый из этих таймеров будет выводить значение счетчика в консоль. Это значит, что значения будут меняться в три раза быстрее!

Для решения описанной проблемы нужно просто сделать так, чтобы повторное нажатие на кнопку не приводило к новому запуску таймера. Например, можно после старта таймера отвязывать от кнопки привязанный обработчик клика:

```

1  let start = document.querySelector('#start');
2
3  start.addEventListener('click', function func() {
4      let i = 0;
5
6      setInterval(function() {
7          console.log(++i);
8      }, 1000);
9
10     this.removeEventListener('click', func); // отвязываем обработчик
11 });

```

©jsPmTrML

## № 1

Возьмите ваше решение предыдущей задачи. Проверьте, что многократное нажатие на кнопку приводит к ускорению отсчета. Исправьте эту проблему.

# Кнопки для запуска и остановки таймера на JavaScript

Давайте теперь сделаем две кнопки: по нажатию на первую пусть таймер запускается, а по нажатию на вторую - останавливается. Здесь уже все будет не так просто и нас ждет подвох. Для того, чтобы понять суть этого подвоха, распишу создание кода по шагам.

Итак, делаем две кнопочки:

```

1  <button id="start">start</button>
2  <button id="stop">stop</button>

```

Получаем ссылки на эти кнопки в переменные:

```

1  let start = document.querySelector('#start');
2  let stop  = document.querySelector('#stop');

```

По нажатию на первую кнопку стартуем таймер, записав его номер в переменную:

```

1  start.addEventListener('click', function() {
2      let timerId = setInterval(function() {
3          console.log('!')
4      }, 1000);
5  });

```

А теперь остановим таймер по нажатию на вторую кнопку:

```

1 start.addEventListener('click', function() {
2   let timerId = setInterval(function() {
3     console.log('!')
4   }, 1000);
5 });
6
7 // Останавливаем таймер:
8 stop.addEventListener('click', function() {
9   clearInterval(timerId);
10 });

```

Однако, если попытаться запустить приведенный выше код, нас ждет сюрприз: при попытке остановить таймер окажется, что переменная `timerId` равна `undefined`! Почему так произошло? Потому что при запуске таймера мы сделали нашу переменную `timerId` локальной внутри функции, привязанной к кнопке `start`.

Для решения проблемы сделаем переменную `timerId` глобальной - в этом случае она будет доступна как в функции запуска таймера, так и в функции остановки:

```

1 let timerId; // сделаем переменную глобальной
2
3 start.addEventListener('click', function() {
4   timerId = setInterval(function() {
5     console.log('!')
6   }, 1000);
7 });
8
9 stop.addEventListener('click', function() {
10  clearInterval(timerId);
11 });

```

## №1

©jsPmTrSB

Пусть дана переменная, в которой изначально хранится число 100. Даны также две кнопки. По нажатию на первую кнопку запустите таймер, который каждую секунду будет уменьшать значение переменной на 1 и выводить новое значение в консоль. Как только значение переменной достигнет нуля - остановите таймер.

По нажатию на вторую кнопку остановите таймер. Также остановите таймер, если вторая кнопка не была нажата, но значение переменной достигло нуля.

## №2

©jsPmTrSB

Некоторый программист написал код, который по нажатию на кнопку запускает таймер, выводящий в консоль текущий момент времени:

```

1 <input type="submit" id="start" value="start">
2 <input type="submit" id="stop" value="stop">
3
4 let start = document.querySelector('#start');
5 let stop = document.querySelector('#stop');
6
7 start.addEventListener('click', function() {
8   let timerId = setInterval(function() {
9     let date = new Date;
10    console.log(date.getMinutes() + ' ' + date.getSeconds());
11  }, 1000);
12 });
13
14 stop.addEventListener('click', function() {
15   clearInterval(timerId);
16 });

```

После запуска кода, однако, оказалось, что кнопка остановки не работает. Исправьте ошибку автора кода.

### №3

⊗jsPmTrSB

Еще один программист также написал код для решения предыдущей задачи:

```
1 <input type="submit" id="start" value="start">
2 <input type="submit" id="stop" value="stop">

1 let start = document.querySelector('#start');
2 let stop = document.querySelector('#stop');
3
4 let timerId;
5
6 start.addEventListener('click', function() {
7   let timerId = setInterval(function() {
8     let date = new Date;
9     console.log(date.getMinutes() + ' ' + date.getSeconds());
10  }, 1000);
11 });
12
13 stop.addEventListener('click', function() {
14   clearInterval(timerId);
15 });
```

После запуска кода, однако, опять оказалось, что кнопка остановки не работает. Исправьте ошибку автора кода.

### №4

⊗jsPmTrSB

Еще один программист также написал код для решения предыдущей задачи:

```
1 <input type="submit" id="start" value="start">
2 <input type="submit" id="stop" value="stop">

1 let start = document.querySelector('start');
2 let stop = document.querySelector('stop');
3
4 let timerId;
5
6 start.addEventListener('click', function() {
7   setInterval(function() {
8     let date = new Date;
9     console.log(date.getMinutes() + ' ' + date.getSeconds());
10  }, 1000);
11 });
12
13 stop.addEventListener('click', function() {
14   clearInterval(timerId);
15 });
```

После запуска кода, однако, опять оказалось, что кнопка остановки не работает. Исправьте ошибку автора кода

Еще один программист также написал код для решения предыдущей задачи:

```
1 <input type="submit" id="start" value="start">
2 <input type="submit" id="stop" value="stop">

1 let start = document.querySelector('start');
2 let stop = document.querySelector('stop');
3
4 let timerId;
5
6 start.addEventListener('click', function() {
7   let timerId = setInterval(function() {
8     let date = new Date;
9     console.log(date.getMinutes() + ' ' + date.getSeconds());
10  }, 1000);
11 });
12
13 stop.addEventListener('click', function() {
14   clearInterval();
15 });
```

После запуска кода, однако, опять оказалось, что кнопка остановки не работает. Исправьте ошибку автора кода.

## №6

©jsPmTrSB

Приведенный мною в теоретической части код не учитывает то, что на кнопку старт можно сделать несколько нажатий. Для исправления этой проблемы можно по нажатию на кнопку старт отвязывать событие от этой кнопки, а по нажатию на кнопку стоп - привязывать обратно. Исправьте проблему.

## Практика на таймеры и работу с DOM в JavaScript

Пусть у нас дан инпут:

```
1 <input id="elem" value="1">
```

Давайте сделаем таймер, который каждую секунду увеличивать значение атрибута value нашего инпута на единицу:

```
1 let elem = document.querySelector('#elem');
2
3 setInterval(function() {
4   elem.value = Number(elem.value) + 1;
5 }, 1000);
```

## №1

©jsPmTrDm

Объясните, зачем в приведенном выше коде используется функция Number. Что будет, если не написать эту функцию в данном коде?

## №2

©jsPmTrDm

Пусть в инпуте в атрибуте value изначально записано число 10. Запустите таймер, который каждую секунду будет уменьшать это число на единицу.

Модифицируйте предыдущую задачу так, чтобы, как только содержимое инпута станет равно нулю, таймер прекратил свою работу.

## Таймеры и потеря контекста в JavaScript

При использовании таймеров в обработчиках событий нас поджидают проблемы с потерей контекста. Давайте посмотрим на примере.

Пусть у нас есть инпут:

```
1 | <input id="elem" value="text">
```

Пусть по клику на этот инпут сработает анонимная функция и внутри этой функции запустится таймер, каждую секунду выводящий что-нибудь в консоль:

```
1 | let elem = document.querySelector('#elem');
2 |
3 | elem.addEventListener('click', function() {
4 |   setInterval(function() {
5 |     console.log('!!!'); // что-нибудь выводим в консоль
6 |   }, 1000);
7 | });
```

Пока все работает верно. Но пусть теперь мы хотим выводить в консоль value нашего инпута - нас ждет сюрприз: в консоль будет выводиться undefined:

```
1 | elem.addEventListener('click', function() {
2 |   setInterval(function() {
3 |     console.log(this.value); // будет выводиться undefined
4 |   }, 1000);
5 | });
```

Все дело в том, что у нас получается функция в функции: есть внешняя анонимная функция, которая вызывается по клику и внутренняя анонимная функция, которую запускает таймер. Во внешней функции this указывает на инпут, но во внутренней - нет. Имеет место потеря контекста.

Почему выводится undefined, а не вываливается ошибка в консоль, как это было в предыдущих уроках? Потому что this внутри функции, вызываемой через setInterval, указывает на window.

Это значит, что мы пытаемся прочитать свойство value у объекта window, вот так: window.value, а такого свойства в нем нет, и мы получаем undefined (не ошибку).

Поправим проблему введением self:

```
1 | elem.addEventListener('click', function() {
2 |   let self = this;
3 |
4 |   setInterval(function() {
5 |     console.log(self.value);
6 |   }, 1000);
7 | });
```

Пусть дан такой код:

```
1 | <input type="button" id="elem" value="1">
```

```
1 <input type="button" id="elem" value="1">
```

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   setInterval(function() {
5     this.value = Number(elem.value) + 1;
6   }, 1000);
7 });
```

Автор кода хотел, чтобы по нажатию на кнопку, значение этой кнопки каждую секунду увеличивалось на 1. Однако, по нажатию на кнопку вообще ничего не происходит. Исправьте ошибку автора кода. Напишите текст, в котором вы дадите объяснение автору кода, почему возникла его ошибка.

## Практика на таймеры и работу с DOM в JavaScript

### № 1

©jsPmTrPr

Дана кнопка. Дан абзац, текстом которого является число. По нажатию на кнопку запустите таймер, который каждую секунду будет увеличивать текст абзаца на 1.

### № 2

©jsPmTrPr

Дана кнопка. Дан абзац, текстом которого является число, например, 10. По нажатию на кнопку запустите таймер, который каждую секунду будет уменьшать текст абзаца на 1. Как только значение абзаца станет равно нулю - остановите таймер.

### № 3

©jsPmTrPr

Дан инпут с числом. Сделайте так, чтобы каждую секунду в нем появлялся квадрат того числа, которое в нем записано.

### № 4

©jsPmTrPr

Дан инпут. В него вводится число. По потери фокуса сделайте так, чтобы в абзаце ниже начал тикать обратный отсчет, начиная с введенного числа. Когда отсчет дойдет до нуля - он должен закончиться.

### № 5

©jsPmTrPr

Дан инпут, кнопка и абзац. В инпут вводится какое-то число. По нажатию на кнопку запишите введенное число в текст абзаца и запустите обратный отсчет в абзаце: пусть каждую секунду число в абзаце уменьшается на единицу, пока не дойдет до нуля.

### № 6

©jsPmTrPr

Дан абзац и две кнопки. Сделайте так, чтобы по нажатию на первую кнопку в абзаце начал тикать таймер от 1 до бесконечности, а по нажатию на вторую таймер останавливался.

### № 7

©jsPmTrPr

Дан абзац. Сделайте так, чтобы каждую секунду он менял свой цвет с красного на зеленый и наоборот.

### № 8

©jsPmTrPr

Если выводить на экран каждую секунду текущий момент времени, то можно сделать тикающие часы. Реализуйте такие же часы, как показано ниже в образце:

00:19:02

# Задержка перед выполнением в JavaScript

Давайте теперь разберем функцию `setTimeout`, которая позволяет сделать задержку перед запуском кода. Эта задержка, в отличие от функции `setInterval`, случится только один раз.

Первым параметром `setTimeout` принимает исходный код функции, а вторым - задержку в миллисекундах перед запуском этой функции.

Посмотрим работу функции на каком-нибудь примере. Пусть у нас есть вот такая кнопка:

```
1 | <input type="submit" id="elem">
```

Получим ссылку на нашу кнопку в переменную:

```
1 | let elem = document.querySelector('#elem');
```

Давайте теперь сделаем так, чтобы по нажатию на кнопку на экран вывелся `alert`, но не сразу, а через 3 секунды после нажатия:

```
1 | elem.addEventListener('click', function() {  
2 |     setTimeout(function() {  
3 |         alert('!');  
4 |     }, 3000);  
5 | });
```

№ 1

©jsPmTrDI

Дан абзац. Напишите код, который выведет сообщение в этот абзац через 10 секунд после загрузки страницы.

# Запуск таймера в JavaScript

Хотя функция `setTimeout` и не предназначена для создания таймеров, однако их все равно можно делать с ее помощью, если воспользоваться [рекурсией](#):

# Работа с рекурсией в JavaScript

В программировании есть такое понятие, как *рекурсия* - это когда функция вызывает сама себя. Давайте посмотрим на примере. Выведем с помощью рекурсии числа от 1 до 10:

```
1 | let i = 1;  
2 |  
3 | function func(){  
4 |     console.log(i);  
5 |     i++;  
6 |  
7 |     if (i <= 10){  
8 |         func(); // здесь функция вызывает сама себя  
9 |     }  
10 | }  
11 | func();
```

Давайте обсудим, как работает этот код.

У нас есть глобальная переменная `i` и функция `func`, внутри которой в консоль выводится содержимое переменной `i`, а затем делается `++`.

Если наша переменная `i` меньше или равна 10, то функция вызывается повторно. Так как переменная `i` - глобальная, то при каждом новом вызове функции в ней будет заданное при предыдущем вызове значение переменной `i`.

Получится, что функция будет вызывать сама себя до тех пор, пока `i` не станет больше 10.

Учтите, что в нашем случае нельзя функцию запустить без `if` - если это сделать, то получится бесконечный вызов функций.

## Запуск таймера в JavaScript

Хотя функция `setTimeout` и не предназначена для создания таймеров, однако их все равно можно делать с ее помощью, если воспользоваться [рекурсией](#):

```
1 let i = 0;
2
3 function timer() {
4   setTimeout(function() {
5     console.log(++i);
6
7     timer(); // вызовем сами себя
8   }, 1000);
9 }
10 timer();
```

Остановить такой таймер можно просто не дав случится рекурсии:

```
1 let i = 0;
2
3 function timer() {
4   setTimeout(function() {
5     console.log(++i);
6
7     if (i < 10) { // запускаем, только если счетчик меньше 10
8       timer();
9     }
10   }, 1000);
11 }
12 timer();
```

№ 1

©jsPmTrDT

Выведите в консоль число 0. Через секунду выведите число 1, через две секунды выведите число 2, через 3 секунды выведите число 3. И так далее до бесконечности.